

PROJECT DOCUMENTATION REPORT

Talk2Mind

A Multimodal AI-Based Mental Well-Being Assessment and Support System

Developed / Implemented By

Bhautik Gondaliya

GitHub Repository: <https://github.com/bhautik2005/Talk2Mind>

Project Demo Video:

<https://drive.google.com/file/d/16Qzb4WScI513cPsEaNK6v1vXBVz874KO/view?usp=sharing>

Deep Learning · Multimodal Emotion Recognition · FastAPI · Streamlit

July 2026

Table of Contents

Abstract.....	3
1. Introduction.....	4
1.1 Problem Statement.....	4
1.2 Objectives.....	4
1.3 Scope.....	4
2. Background and Related Work.....	5
2.1 CMU-MOSEI.....	5
2.2 RAVDESS.....	5
2.3 TESS-style Speech Emotion Recordings.....	5
2.4 Deep Learning Architectures Used.....	5
3. System Architecture.....	6
3.1 High-Level Architecture.....	6
3.2 Technology Stack.....	6
4. Dataset Description.....	8
4.1 Dataset 1 — CMU-MOSEI (Text + Audio).....	8
4.2 Dataset 2 — Multimodal Dataset (Video + supplementary Audio).....	8
4.3 Unified Label Space.....	9
5. Model Design and Implementation.....	10
5.1 Model 1 — Text Emotion Model.....	10
5.2 Model 2 — Audio / Speech Emotion Model.....	10
5.3 Model 3 — Video / Facial Emotion Model.....	11
5.4 Shared Embedding Design.....	11
6. Training Methodology.....	13
6.1 Training Environment.....	13
6.2 Performance Optimization.....	13
6.3 Hyperparameters.....	13
6.4 Data Splitting.....	14
7. Results and Evaluation.....	15
7.1 Observed Training Metrics.....	15
7.2 Text and Audio Models — Interpretation.....	15
7.3 Video Model — Case Study: Diagnosing a Model Collapse.....	15
8. Deployment Architecture.....	17
8.1 FastAPI Backend.....	17
8.2 Streamlit Dashboard.....	17
8.3 Deployment Folder Structure.....	18
9. Testing and Validation.....	19
10. Challenges Faced.....	20
10.1 Heterogeneous Label Sources.....	20
10.2 Training Time on Free-Tier Colab.....	20
10.3 Multi-Label Class Imbalance.....	20
10.4 Keeping Two Deliverables in Sync.....	20
11. Future Scope.....	21
12. Conclusion.....	22
13. References.....	23
Appendix A — Combined Repository Folder Structure.....	24
Appendix B — Emotion Label Set.....	24
Appendix C — Document Revision Note.....	24

Abstract

Talk2Mind is a multimodal artificial intelligence system designed to assess a person's mental well-being through three independent, complementary channels of expression: what they say (text), how they say it (voice/speech), and how they look while saying it (facial expression). Each modality is processed by a dedicated deep learning model — a Bidirectional LSTM network for text, a 1D-CNN + LSTM network for speech, and a TimeDistributed CNN + LSTM network for facial video — and the resulting emotion predictions are combined into a single, easy-to-read well-being dashboard.

The system was trained using two complementary datasets: CMU-MOSEI, a large-scale multimodal sentiment and emotion corpus that supplied paired transcripts and speech recordings for the text and audio models, and a second multimodal collection (referred to throughout this report as the Multimodal Dataset) combining RAVDESS-style labeled video recordings, TESS-style labeled speech recordings, and EEG data. The RAVDESS video portion trained the facial emotion model; the EEG data was collected as part of the same dataset but was intentionally left out of scope for this phase of the project.

The trained models were packaged into a production-style application: a FastAPI backend exposing three prediction endpoints (text, audio, video), and a Streamlit front-end that walks a user through a short, roughly five-to-six-minute, three-step check-in before presenting a consolidated well-being dashboard with per-modality gauges, an emotion breakdown chart, and plain-language recommendations.

This report documents the problem motivation, system architecture, dataset handling, model design, training methodology (including a Google Colab pipeline optimized for training speed), evaluation results, a real debugging case study encountered during training, the deployment architecture, and directions for future work.

1. Introduction

1.1 Problem Statement

Mental health issues such as stress, anxiety, depression, sleep disorders, and emotional burnout have increased significantly in recent years. A large proportion of students and working professionals experience mental health challenges but remain unaware of their symptoms until the condition becomes severe. Traditional mental health assessment relies on consultation with a professional, which is often inaccessible, expensive, or simply intimidating for someone taking their first step toward understanding their own emotional state.

Talk2Mind addresses this gap by offering a low-friction, self-guided check-in that a person can complete privately in under six minutes, using nothing more than a short piece of writing, a short voice recording, and a short video clip.

1.2 Objectives

1. Build an AI-powered mental well-being assessment tool that analyzes a person's emotional state through text, speech, and facial expression.
2. Train independent deep learning models per modality using publicly available multimodal emotion datasets (CMU-MOSEI and a secondary multimodal dataset combining RAVDESS video and TESS-style audio).
3. Package the trained models behind a REST API (FastAPI) so they can be reused by any client application.
4. Build an end-user-facing dashboard (Streamlit) that walks a person through a short, structured check-in and reports results in a clear, non-clinical way.
5. Document the engineering trade-offs made along the way — particularly around training-time optimization and diagnosing a real model-collapse issue encountered on the video model — so the work is reproducible and extensible.

1.3 Scope

This phase of the project covers three modalities: text, audio (speech), and video (facial expression). EEG data was present in the secondary dataset but is explicitly out of scope for the current implementation and is noted in Section 11 (Future Scope) as a natural extension. A fourth, fusion-layer model that combines all three modalities into one score was designed conceptually (see the original Talk2Mind concept note) but the delivered implementation trains and serves the three unimodal models independently, combining their outputs at the presentation layer (the Streamlit dashboard) rather than inside a trained fusion network.

2. Background and Related Work

Emotion recognition from a single modality is a mature area of research, but single-channel signals are individually noisy and ambiguous — a flat tone of voice does not necessarily mean a flat emotional state, and the same is true of a neutral facial expression or a short, ambiguous sentence. Multimodal emotion recognition combines two or more channels (text, audio, video, and sometimes physiological signals like EEG) on the premise that channels tend to disagree in informative ways, and that combining them produces a more robust estimate of a person's underlying state.

2.1 CMU-MOSEI

CMU-MOSEI (CMU Multimodal Opinion Sentiment and Emotion Intensity) is a widely used benchmark dataset for multimodal sentiment and emotion research. It consists of short video clips of speakers, each annotated with a transcript, sentiment score, and six discrete emotion labels: happiness, sadness, anger, surprise, disgust, and fear. The dataset is organized into training, validation, and test splits, with associated audio chunks pre-segmented per utterance — the exact structure used in this project (see Section 4.1).

2.2 RAVDESS

The Ryerson Audio-Visual Database of Emotional Speech and Song (RAVDESS) provides video recordings of 24 professional actors vocalizing statements in a neutral North American accent, with emotion, intensity, and other metadata encoded directly into each filename. This makes RAVDESS convenient for programmatic label extraction without needing a separate label file, which is the approach used for the video model in this project.

2.3 TESS-style Speech Emotion Recordings

The secondary dataset's audio component follows the naming convention popularized by the Toronto Emotional Speech Set (TESS): two actresses (an older adult and a younger adult) speaking a fixed carrier phrase, with the target emotion appended as the final underscore-separated token in the filename (for example, sad, angry, neutral). This project's data loader parses that token directly rather than requiring a separate label spreadsheet.

2.4 Deep Learning Architectures Used

- Bidirectional LSTM (BiLSTM): reads a sequence in both forward and backward order, which is well suited to short text where later words can change the meaning of earlier ones.
- 1D Convolutional Neural Network (CNN): slides a learnable filter across a time axis, well suited to detecting short, local acoustic patterns in a spectrogram-like feature map.
- Long Short-Term Memory (LSTM): a recurrent architecture capable of retaining information across longer sequences than a plain RNN, used here to model how emotional cues evolve over the duration of an utterance or a video clip.
- TimeDistributed CNN: applies the same 2D CNN independently to every frame of a video sequence before an LSTM aggregates the resulting per-frame features over time.

3. System Architecture

3.1 High-Level Architecture

The system follows a linear pipeline per modality: raw input is captured, converted into a fixed-size numerical feature representation, passed through a modality-specific deep learning model, and the resulting emotion-probability vector is displayed to the user. The three modalities run independently and are combined only at the presentation layer.

Data Acquisition		Feature Extraction		Model		Output
-----		-----		-----		-----
Typed text	--->	Tokenization (Text-Vectorization, fixed vocabulary)		BiLSTM + ANN	--->	6-class emotion probabilities
Recorded / uploaded audio (.wav)	--->	MFCC + log-mel + RMS energy, fixed length, z-normalized		1D-CNN + LSTM	--->	6-class emotion probabilities
Recorded / uploaded video (.mp4)	--->	Frame sampling (OpenCV) + face crop, fixed frame count and resolution		TimeDistributed CNN + LSTM	--->	6-class emotion probabilities

Figure 3.1 — Per-modality processing pipeline, from raw input to emotion probabilities.

At the presentation layer, the three probability vectors are converted into a 0-100 well-being score per modality (see Section 8.2) and shown alongside an overall combined score and a plain-language recommendation.

3.2 Technology Stack

Layer	Technology	Purpose
Model training	TensorFlow / Keras	Building and training the text, audio, and video deep learning models
Audio feature extraction	librosa, soundfile	MFCC, log-mel spectrogram, and RMS energy extraction from .wav files
Video feature extraction	OpenCV (cv2)	Frame sampling and Haar-cascade face detection/cropping
Data handling	pandas, numpy, scikit-learn	CSV loading, label engineering, train/validation splitting
Training environment	Google Colab (GPU runtime)	Free GPU access for training all three models
Backend API	FastAPI, Uvicorn	Serving trained models behind REST endpoints
Front-end dashboard	Streamlit, Plotly	3-step guided check-in UI and results dashboard
Persistence	Google Drive (training), local	Storing trained model artifacts (.h5 files) and

Layer	Technology	Purpose
	filesystem (serving)	vocabulary

4. Dataset Description

Two datasets were used, each supplying different modalities. The data loaders were written to auto-detect CSV column names and to parse labels directly from filenames where no separate label file exists, minimizing the amount of manual re-mapping required when either dataset is refreshed or replaced.

4.1 Dataset 1 — CMU-MOSEI (Text + Audio)

CMU-MOSEI supplies paired transcripts and pre-segmented audio chunks, organized into Train / Validation / Test splits, plus a Labels folder of CSV files carrying the transcript text and six emotion columns (happy, sad, angry, surprise, disgust, fear) per utterance.

```
CMU-MOSEI/
├── Audio_chunk/
│   ├── Train_modified/      .wav audio chunks (training split)
│   ├── Val_modified/        .wav audio chunks (validation split)
│   ├── Test_modified/       .wav audio chunks (test split)
│   ├── Test_original/       original (pre-fix) test chunks
│   └── Val_original/         original (pre-fix) validation chunks
└── Labels/
    ├── Data_Train_modified.csv
    ├── Data_Val_modified.csv
    ├── Data_Test_modified.csv
    └── Data_Test_original_without_neg_time.csv
```

Figure 4.1 — CMU-MOSEI folder structure as used in this project.

Each audio chunk filename encodes the source video identifier and the exact start/end timestamp of the utterance, e.g. -9y-fZ3swSY_0.0000_6.3680.wav. The data loader reconstructs this filename directly from the CSV row (video_id, start, end) to locate the matching audio file, rather than relying on a separate join key — this was verified against the real directory listing before being used for training.

4.2 Dataset 2 — Multimodal Dataset (Video + supplementary Audio)

The second dataset bundles three sub-collections. Only the video sub-collection was used for model training in the delivered implementation; the audio sub-collection was evaluated as a candidate addition to the audio model and ultimately excluded to keep training time manageable (see Section 6.2); the EEG sub-collection was not used.

```
Multimodal_Dataset/
├── Audio_Dataset/Audio_Dataset/
│   ├── Depression/Stage1/    TESS-style .wav, emotion in filename suffix
│   ├── Depression/Stage2/    TESS-style .wav, emotion in filename suffix
│   └── Normal/               TESS-style .wav, emotion in filename suffix
├── EEG_Data/EEG_Data/       .mat files – out of scope for this phase
└── Video_Dataset/Video_Dataset/
    ├── Video_Speech_Actor_01 .. 24 / Actor_NN/
    │   └── 01-01-EE-II-SS-RR-AA.mp4  RAVDESS-encoded filename
```

Figure 4.2 — Multimodal Dataset folder structure as used in this project.

RAVDESS filenames encode seven fields separated by hyphens: modality, vocal channel, emotion, intensity, statement, repetition, and actor number. The emotion field (third position) was parsed directly to build the video model's labels, using the mapping in Table 4.1.

RAVDESS Code	Meaning	Mapped Talk2Mind Label
01	neutral	excluded (all-zero / no strong emotion)
02	calm	excluded (all-zero / no strong emotion)
03	happy	happy
04	sad	sad
05	angry	angry
06	fearful	fear
07	disgust	disgust
08	surprised	surprise

Table 4.1 — RAVDESS emotion code mapping used by the video data loader.

The TESS-style audio filenames (Depression/Stage1, Stage2, Normal folders) encode the emotion as the final underscore-separated token, e.g. OAF_back_sad.wav or OAF_back_neutral.wav. This convention was parsed with the same six-label mapping, with the token "ps" (pleasant surprise) mapped to surprise and "neutral" mapped to an all-zero label vector.

4.3 Unified Label Space

A key design decision was to standardize all three modalities onto the same six-class emotion label space — happy, sad, angry, surprise, disgust, fear — even though the two source datasets label their data in structurally different ways (CMU-MOSEI via annotated CSV columns, the secondary dataset via filename encoding). This makes it possible to reuse a single Streamlit dashboard and a single set of emotion-label constants across all three backend inference modules without per-modality special-casing.

5. Model Design and Implementation

All three models share a common output design — a sigmoid layer over the six emotion classes, trained with binary cross-entropy loss — which treats emotion recognition as a multi-label problem (a person's expression can plausibly carry more than one emotion at once) rather than a mutually-exclusive single-label classification problem.

5.1 Model 1 — Text Emotion Model

Purpose: process typed check-in responses or conversational transcripts and estimate an emotion/risk profile from the language used.

Layer	Configuration
Input	Tokenized integer sequence, max length 50
Embedding	Vocabulary size 20,000 → 128-dimensional embedding, mask_zero=True
Bidirectional LSTM (1)	128 units, return_sequences=True
Bidirectional LSTM (2)	64 units
Dense (ANN head)	256 units, ReLU, Dropout 0.3
Embedding output	128-dimensional "text_emotion_embedding" (ReLU)
Output	6-unit sigmoid — one probability per emotion

Table 5.1 — Text model architecture.

The vocabulary is built with a Keras TextVectorization layer fit on the training split's transcripts only. The fitted vocabulary is exported to text_vectorizer_vocab.pkl alongside the trained model, since a saved .h5 file does not itself retain the tokenizer state — the serving application reconstructs an identical TextVectorization layer from this file at inference time.

5.2 Model 2 — Audio / Speech Emotion Model

Purpose: analyze short speech recordings for emotional cues carried in pitch, energy, tempo, and timbre.

Layer	Configuration
Input	(time_steps, feature_dim) — MFCC + log-mel + RMS energy, z-normalized
Conv1D (1)	64 filters, kernel 5, ReLU, BatchNorm, MaxPool 2
Conv1D (2)	128 filters, kernel 5, ReLU, BatchNorm, MaxPool 2
Conv1D (3)	256 filters, kernel 3, ReLU, BatchNorm, Dropout 0.3
LSTM (1)	128 units, return_sequences=True
LSTM (2)	64 units
Dense (ANN head)	128 units, ReLU, Dropout 0.3

Layer	Configuration
Embedding output	128-dimensional "speech_emotion_embedding" (ReLU)
Output	6-unit sigmoid — one probability per emotion

Table 5.2 — Audio model architecture.

Audio features are extracted with librosa: 40 MFCC coefficients, a 40-band log-mel spectrogram, and RMS energy, concatenated along the feature axis and padded/truncated to a fixed number of time steps so that batches can be stacked. An earlier version of the pipeline also extracted pitch (fundamental frequency) via librosa's pyin algorithm; this was removed after benchmarking showed it cost roughly 70 times longer per file than the rest of the feature set combined, for negligible benefit to this task (see Section 6.2).

5.3 Model 3 — Video / Facial Emotion Model

Purpose: track facial expression across a short video clip to estimate the speaker's emotional state visually.

Layer	Configuration
Input	(num_frames, H, W, 3) sequence of face-cropped RGB frames
Per-frame CNN backbone	3× Conv2D (32/64/128 filters) + BatchNorm + MaxPool, GlobalAveragePooling2D
TimeDistributed wrapper	Applies the CNN backbone independently to every frame
LSTM (1)	128 units, return_sequences=True — aggregates frame-level features over time
LSTM (2)	64 units
Dense (ANN head)	128 units, ReLU, Dropout 0.3
Embedding output	128-dimensional "visual_emotion_embedding" (ReLU)
Output	6-unit sigmoid — one probability per emotion

Table 5.3 — Video model architecture.

Frames are sampled at evenly spaced intervals across the clip, and the largest detected face per frame (via OpenCV's Haar cascade) is cropped and resized before being stacked into a fixed-length sequence. A frozen, ImageNet-pretrained EfficientNetB0 backbone was also implemented as a drop-in alternative to the custom CNN, matching the ResNet/EfficientNet visual feature extraction referenced in the original architecture diagram, for use once the lighter custom-CNN pipeline has been validated end-to-end.

5.4 Shared Embedding Design

Each of the three models exposes a named 128-dimensional embedding layer (text_emotion_embedding, speech_emotion_embedding, visual_emotion_embedding) immediately before its final classification layer. This was a deliberate design choice: even though the delivered

system combines modality outputs at the presentation layer rather than through a trained fusion network, these embeddings mean a genuine late-fusion model — concatenating all three vectors and passing them through a small dense network to predict one overall well-being score — can be added later without retraining the three base models from scratch.

6. Training Methodology

6.1 Training Environment

All three models were trained in Google Colab, using a T4 GPU runtime. The dataset was staged in Google Drive under a single parent folder, `Talk2Mind_data/`, with the two source datasets (CMU-MOSEI and Multimodal_Dataset) as subfolders. Trained model artifacts were written back to Drive so they persist across Colab runtime disconnects.

6.2 Performance Optimization

An initial training pipeline combined audio from both datasets and included pitch extraction in the audio feature set. Benchmarking showed two significant bottlenecks, both of which were addressed in a revised, faster pipeline:

- Pitch extraction (`librosa.pyin`) cost approximately 0.65 seconds per audio file, versus roughly 0.01 seconds for MFCC, log-mel, and RMS energy combined — around 70 times slower for one feature block. It was removed from the feature set.
- Every audio and video feature was being recomputed from the raw file on every training epoch. A `tf.data .cache()` step was added to each pipeline so that feature extraction runs once, on the first epoch, and is reused directly from memory/disk on every subsequent epoch.

Additional reductions applied in the faster pipeline: audio clip duration was reduced from 4.0 to 3.0 seconds, the mel-spectrogram resolution was reduced from 64 to 40 bands, video frame count was reduced from 16 to 8 frames per clip, and video frame resolution was reduced from 96×96 to 64×64 pixels. The audio model's training data was also simplified to draw from CMU-MOSEI only, rather than combining it with the secondary dataset's audio, further reducing both data volume and pipeline complexity.

A configurable `MAX_SAMPLES_PER_SPLIT` cap was also added to the training notebook, allowing a full end-to-end smoke test to run in a few minutes on a small subset of data before committing to a full-scale run.

6.3 Hyperparameters

Setting	Text Model	Audio Model	Video Model
Batch size	16	16	16
Max epochs	10	10	8
Early stopping patience	3 epochs	3 epochs	3 epochs
Optimizer	Adam	Adam	Adam
Learning rate	1e-4	1e-4	1e-4
Loss function	Binary cross-entropy	Binary cross-entropy	Binary cross-entropy

Table 6.1 — Training hyperparameters used across all three models.

6.4 Data Splitting

The text and audio models use the Train / Validation / Test splits already defined by CMU-MOSEI's own CSV files. The video model, which draws from RAVDESS-style filenames with no predefined split, uses an 85/15 random train/validation split generated with scikit-learn's `train_test_split`.

7. Results and Evaluation

This section reports training results as observed during an actual training run in the Colab pipeline described in Section 6, and documents a real issue diagnosed in the video model's results together with the fix applied.

7.1 Observed Training Metrics

Model	Epoch	Train Accuracy	Train AUC	Train Loss	Val Accuracy	Val AUC	Val Loss
Text	10/100	0.7413	0.8499	0.2730	0.6760	0.7508	0.3478
Audio	7/100	0.7302	0.7573	0.3019	0.6871	0.7448	0.3388
Video	59/100	0.2990	0.0000	0.0016	0.5278	0.0000	0.0003

Table 7.1 — Representative epoch snapshots from an actual training run of all three models.

7.2 Text and Audio Models — Interpretation

The text and audio models both reached a plausible working state: validation AUC around 0.74-0.75 and validation accuracy in the high 0.60s to high 0.80s range indicates the models are learning a genuine, generalizing signal rather than memorizing the training set, though there is room for improvement before the models would be considered production-grade. Two concrete next steps were identified for improving these numbers further:

- Confirm whether the underlying CMU-MOSEI emotion columns are already binary (0/1) or use the dataset's standard Likert-style intensity scale (0-3); if the latter, binarizing the labels (presence/absence rather than raw intensity) before training against a sigmoid/binary cross-entropy objective is expected to improve fit quality, since the loss function's assumptions currently do not match a multi-level target.
- Replace the default string metric "accuracy" with an explicit `tf.keras.metrics.BinaryAccuracy`, and set the AUC metric's `multi_label` flag, so that reported numbers are computed the way they are intended to be interpreted for a multi-label sigmoid output.

7.3 Video Model — Case Study: Diagnosing a Model Collapse

The video model's numbers in Table 7.1 show a distinctive failure pattern: training loss collapsing to a very small value (0.0016) while both accuracy and AUC remain poor, with AUC reported as exactly 0.0. This combination is a signature of a model that has learned to output near-zero for every class on every input, rather than actually discriminating between emotions — a degenerate but loss-minimizing solution when the training labels are heavily skewed toward all-zero.

Root Cause

The RAVDESS video dataset encodes eight emotion categories per Table 4.1, but only six map onto Talk2Mind's target label space. The remaining two categories — neutral and calm — were, in the original pipeline, mapped to an all-zero label vector rather than being excluded. Since neutral and calm together account for roughly a quarter of the RAVDESS dataset, and every genuinely emotional clip is itself sparsely labeled (only one of six positions set to 1), the overall proportion of positive

labels seen during training was low enough that a model minimizing binary cross-entropy could achieve a very low loss simply by predicting close to zero everywhere, without learning to discriminate at all.

Fix Applied

6. Filter out any video clip whose parsed label vector sums to zero (i.e., drop neutral and calm clips) before building the train/validation split, since these clips carry no signal for a six-class emotion classifier.
7. Stratify the train/validation split by the dominant emotion label (using scikit-learn's stratify parameter) so that every emotion class is represented in both the training and validation sets, rather than risking a rare class being absent from validation entirely.
8. Replace the ambiguous default metrics with `tf.keras.metrics.BinaryAccuracy` and an explicitly multi-label AUC metric, and switch the early-stopping monitor from `val_loss` to `val_auc`, since a shrinking loss on a class-imbalanced multi-label problem does not reliably indicate improving discrimination.

This case study is included in the report because it reflects a genuine debugging pass performed during development, and because the underlying cause — silently mapping unused label categories to an all-zero vector rather than excluding them — is a common and easy-to-miss source of model collapse in any multi-label emotion recognition pipeline built on a dataset with more source categories than target categories.

8. Deployment Architecture

The trained models are served through a two-tier local application: a FastAPI backend that owns model loading and inference, and a Streamlit front-end that owns the guided check-in experience and result visualization. The two tiers communicate over plain HTTP, which keeps them independently deployable and testable.

8.1 FastAPI Backend

The backend loads each of the three .h5 models lazily (on first request) and exposes one prediction endpoint per modality, plus a health-check endpoint that reports whether all required model files are present at their configured paths.

Endpoint	Method	Input	Output
/health	GET	—	Boolean found/not-found status for each of the 3 model files and the text vocabulary file
/predict/text	POST	JSON { "text": "..." }	6-class emotion probability dictionary
/predict/audio	POST	Multipart file upload (.wav)	6-class emotion probability dictionary
/predict/video	POST	Multipart file upload (.mp4/.mov)	6-class emotion probability dictionary

Table 8.1 — FastAPI endpoint summary.

All model file locations are centralized in a single configuration module (`model_paths.py`), so pointing the application at a newly retrained set of models is a one-file change. The text model additionally requires an exported vocabulary file (`text_vectorizer_vocab.pkl`), since a saved .h5 file does not retain the `TextVectorization` layer's learned vocabulary.

8.2 Streamlit Dashboard

The front-end guides the user through a linear, three-step check-in designed to take roughly five to six minutes in total, followed by a combined results dashboard:

9. Text check-in (~1-2 minutes) — the user types a free-text response to a reflective prompt; the response is sent to `/predict/text`.
10. Voice check-in (~1-2 minutes) — the user records or uploads a short .wav clip responding to a spoken prompt; sent to `/predict/audio`.
11. Face check-in (~1-2 minutes) — the user uploads a short video clip; sent to `/predict/video`.
12. Dashboard — the three sets of emotion probabilities are converted into a 0-100 well-being score per modality using a simple, clearly-isolated scoring function, displayed as gauge charts alongside an overall combined score, a per-emotion breakdown chart, and a plain-language recommendation.

The per-modality-to-score conversion (`distress_score` in the Streamlit app) is intentionally kept as an isolated, easily replaceable function, since the exact formula for turning raw emotion probabilities into a single well-being number is a product decision that is expected to evolve independently of the underlying models.

8.3 Deployment Folder Structure

```
talk2mind_app/
├── models/                trained .h5 files + text vocabulary go here
├── backend/
│   ├── model_paths.py    single file to edit for model file locations
│   ├── main.py           FastAPI app and endpoint definitions
│   ├── inference/        text_infer.py / audio_infer.py / video_infer.py
│   └── preprocessing/    audio_preprocessing.py / video_preprocessing.py
├── frontend/
│   └── streamlit_app.py   3-step guided check-in + results dashboard
├── run_backend.bat / .sh
└── run_frontend.bat / .sh
```

Figure 8.1 — Deployment application folder structure.

9. Testing and Validation

Two levels of testing were applied during development. First, every generated Python module was syntax-checked (`py_compile`) and, where applicable, imported and exercised directly to confirm that model-building functions produce the expected tensor shapes and that a FastAPI application built from the code exposes the intended routes without runtime errors.

Second, and more substantively, the full Colab training pipeline — dataset loading, filename-based label parsing, feature extraction, `tf.data` pipeline construction, model training, and model saving — was validated end-to-end against a small synthetic dataset constructed to mirror the real folder structure and filename conventions exactly (matching CMU-MOSEI's `audio_id_start_end.wav` pattern, RAVDESS's hyphen-separated emotion coding, and the TESS-style underscore-separated emotion suffix). This caught two real issues before they reached the user: a directory-creation bug caused by shell brace-expansion being suppressed inside quoted strings, and confirmation that the feature-caching change actually produced the intended speed-up (an audio training epoch dropped from roughly 8 seconds to 27 milliseconds once cached).

This synthetic-data validation approach is recommended as a standing practice for any future changes to the data loading or feature extraction code: it exercises the full pipeline in well under a minute, without requiring the real, multi-gigabyte datasets to be present.

10. Challenges Faced

10.1 Heterogeneous Label Sources

CMU-MOSEI provides emotion labels as annotated CSV columns, while the secondary dataset encodes emotion directly into filenames using two different conventions (RAVDESS numeric codes for video, TESS-style text suffixes for audio). Building a single shared six-class label space that both sources could be mapped onto — without silently discarding information or, as in the video model's case, silently injecting a large number of empty labels — required careful, explicit mapping tables rather than a generic loader.

10.2 Training Time on Free-Tier Colab

Colab's free-tier GPU runtime is time-limited and shared, making naive feature extraction (recomputing MFCC, spectrograms, and pitch from raw audio on every single epoch) impractical at dataset scale. Profiling individual feature-extraction calls to find the actual bottleneck (pitch extraction, in this case) rather than guessing was necessary to get a meaningful speed-up rather than a marginal one.

10.3 Multi-Label Class Imbalance

Because several source emotion categories do not map onto the target six-class space (RAVDESS's neutral and calm), and because true multi-label emotion data is naturally sparse (most positions are 0 for any given sample), a naive training setup is vulnerable to the kind of model collapse documented in Section 7.3. Diagnosing this required looking past the reported loss value — which appeared to be improving — to the accuracy and AUC metrics, which told the true story.

10.4 Keeping Two Deliverables in Sync

Because the training pipeline (Colab notebook) and the serving pipeline (FastAPI app) are necessarily separate codebases, feature-extraction logic (audio duration, mel-band count, video frame count/resolution) has to be kept identical between the two, or a trained model's input expectations will silently mismatch what the serving application feeds it at inference time. This is called out explicitly in both codebases' comments and in the deployment README.

11. Future Scope

13. Trained late-fusion model: use the three 128-dimensional embedding layers already exposed by each model (text_emotion_embedding, speech_emotion_embedding, visual_emotion_embedding) to train a genuine fusion network that predicts a single well-being score, rather than combining modality outputs only at the presentation layer.
14. EEG integration: the secondary dataset already includes EEG recordings (.mat files) that were out of scope for this phase; a fourth modality branch could be added following the same embedding-extraction pattern used by the other three models.
15. Live webcam capture: the current video check-in requires an uploaded clip; integrating streamlit-webrtc would allow in-browser recording directly from a webcam.
16. Persistent storage and accounts: the current application is single-session and in-memory only; adding a database layer (for example SQLite or PostgreSQL) and basic authentication would allow a user to track their well-being trend over multiple check-ins over time, matching the original concept note's goal of progress tracking.
17. Stronger visual backbone: swap the lightweight custom CNN video backbone for the frozen EfficientNetB0 backbone already implemented as an option, once training-time budget allows.
18. Clinical validation: before any real-world deployment in a mental-health-adjacent context, the well-being scoring formula and model outputs would need review and validation by a qualified mental health professional; this project is a self-assessment aid, not a diagnostic tool.

12. Conclusion

Talk2Mind demonstrates an end-to-end multimodal emotion recognition pipeline, from raw, heterogeneously labeled datasets through three independently trained deep learning models to a deployable REST API and a guided, user-facing dashboard. The project's engineering value lies not only in the model architectures themselves — which follow well-established patterns for text, audio, and video emotion recognition — but in the practical decisions made to get there reliably: reconciling two datasets with fundamentally different labeling conventions onto one shared label space, profiling and removing a training-time bottleneck that was seventy times more expensive than necessary, and diagnosing a genuine model-collapse failure by reading past a misleadingly low loss value to the metrics that actually reflected the model's behavior.

The result is a working, documented, and reproducible system: a Colab notebook that trains all three models from the two source datasets, a FastAPI + Streamlit application that serves them, and this report, which records both what was built and why specific design and debugging decisions were made along the way.

13. References

- [1] Zadeh, A., Liang, P. P., Poria, S., Cambria, E., & Morency, L.-P. Multimodal Language Analysis in the Wild: CMU-MOSEI Dataset and Interpretable Dynamic Fusion Graph. Association for Computational Linguistics (ACL), 2018.
- [2] Livingstone, S. R., & Russo, F. A. The Ryerson Audio-Visual Database of Emotional Speech and Song (RAVDESS): A dynamic, multimodal set of facial and vocal expressions in North American English. PLoS ONE, 2018.
- [3] Pichora-Fuller, M. K., & Dupuis, K. Toronto Emotional Speech Set (TESS). University of Toronto, Psychology Department.
- [4] Hochreiter, S., & Schmidhuber, J. Long Short-Term Memory. Neural Computation, 1997.
- [5] TensorFlow / Keras documentation — <https://www.tensorflow.org>
- [6] librosa: Audio and Music Signal Analysis in Python — <https://librosa.org>
- [7] FastAPI documentation — <https://fastapi.tiangolo.com>
- [8] Streamlit documentation — <https://docs.streamlit.io>

Appendix A — Combined Repository Folder Structure

```
talk2mind/                                (model training project)
├── config.py
├── data/dataset_loader.py
├── preprocessing/
│   ├── audio_preprocessing.py
│   └── video_preprocessing.py
├── models/
│   ├── text_model.py
│   ├── audio_model.py
│   ├── video_model.py
│   └── fusion_model.py
├── train.py
└── Talk2Mind_Train_3Models_Fast.ipynb    (Colab training notebook)

talk2mind_app/                            (deployment application)
├── models/
├── backend/ (FastAPI)
└── frontend/ (Streamlit)
```

Appendix B — Emotion Label Set

All three models, and the combined dashboard, share the same fixed six-class emotion label order:

```
["happy", "sad", "angry", "surprise", "disgust", "fear"]
```

Appendix C — Document Revision Note

This report was compiled from the project's implementation history, training logs, and the documented debugging session for the video model, at the request of the project's developer, Bhautik Gondaliya.